

ArenaAI Fare Rules - Codebase Audit

Part 1 — Code-grounded audit of fare rules, services, data sources, and observed risks

Repository: github.com/tarun1sisodia/ArenaAI • Branch: main

Scope and confidence Reviewed the public main-branch source files and repository fare audit documents. This is a static code review, not a production quote run. Findings below separate directly inspected implementation from risks described by repository documentation.

Executive summary

ArenaAI has a real backend fare engine and a good foundation for server-authoritative booking/payment, but the commercial pricing model is not consistently represented across the customer website. Fare behavior is split between admin-editable PostgreSQL rules, fixed product records, legacy route catalogues, hardcoded frontend display prices, and client-side indicative calculators.

- Backend quote path: POST /api/v1/fares/calculate -> validation/schema -> fare service -> product and distance resolvers -> pure fare engine -> calculator/strategy -> finalizer -> response.
- Booking/payment integrity: booking draft creation recalculates on the server; payment checkout derives the payment amount from the stored booking advance rather than trusting the browser.
- Largest business risk: the displayed customer price can differ from the backend quote after an admin fare change, because homepage, route, monument, package, and booking browse screens still have static or client-calculated values.
- Largest algorithmic complexity: distinct rules for standard cabs vs commercial group vehicles, a hard 300 km boundary, fixed package prices, local packages, transfer rates, night charges, driver allowance, tolls, promos, and advance calculations.
- Immediate priority: establish one canonical server quote contract and unify all customer price displays behind it; do not simply change per-km values before defining which product types use per-km versus fixed pricing.

1. Key code map

Concern	Main files inspected	Role
Service/orchestration	backend/src/modules/fares/fare.service.ts	Loads product overrides, resolves billable distance, gets promo context, calls pure engine, reapplies validated DB promo.
Fare dispatch and validations	backend/src/modules/fares/fare.engine.ts	Validates distance/date bounds; routes to dossier/package/local/transfer/route paths and strategy.
Fleet and legacy catalogue	backend/src/modules/fares/fare.catalogue.ts	Static vehicle defaults, static route/package/local/transfer prices, fallback distance map, tier aliases.
Vehicle strategy selection	backend/src/modules/fares/fare.strategy.ts	Dispatches standard fleet versus Tempo/Urbania commercial strategy.
Standard outstation	backend/src/modules/fares/calculators/route.calculator.ts	One-way rates, same-day round-trip multiplier, multiday minimums and driver allowance.

Concern	Main files inspected	Role
Tempo/Urbania exceptions	backend/src/modules/fares/calculators/commercial-group.calculator.ts	Below-300-km doubled distance, daily driver allowance, promo restriction.
Fixed tour packages	backend/src/modules/fares/calculators/tour-package.calculator.ts	Admin fleetPrices precedence, starting price plus tier upgrade, legacy fixed-price path.
Local sightseeing	backend/src/modules/fares/calculators/local-tour.calculator.ts	Catalog local package prices or fixed 8-hour/80-km and 12-hour/120-km package prices.
Airport/station transfer	backend/src/modules/fares/calculators/transfer.calculator.ts	Special static transfer lookup or database catalog ride pricing.
Finalization	backend/src/modules/fares/calculators/fare.finalizer.ts	Adds eligible night/toll/driver costs, applies promo, computes total/advance/balance and response rules.
Distance resolution	backend/src/modules/fares/resolvers/distance.resolver.ts; route.resolver.ts	Distance precedence plus route lookup/reverse route/fallback estimation.
Product/rule resolution	backend/src/modules/fares/resolvers/product.resolver.ts	Reads published tour packages, transfer routes, local packages, route records and active fare rules.
Fleet API	backend/src/modules/fares/fleet/fleet.service.ts; fare.controller.ts; fare.routes.ts	Exposes live fleet specifications and per-km overrides at GET /api/v1/fleet.
Admin editor and persistence	admin/src/pages/FaresPage.tsx; backend/src/modules/admin/admin.service.ts	Edits five fleet tiers/outstation config, stores new ruleset version, audit log, triggers frontend rebuild.
Frontend indicative logic	react/src/fares.ts; react/src/utls/customDistance.ts; react/src/data/prices.ts	Static route/distance fallbacks, indicative totals, fixed local/package prices, frontend advance estimates.
Prior audit docs	docs/PRICE_FLOW_AUDIT.md; docs/PRICE_SOURCE_MATRIX.md; docs/CUSTOMER_RENDERING_AUDIT.md; docs/MIGRATION_PLAN.md	Repository's recorded price inventory and the planned normalization sequence.

2. How a fare is computed today

1. Validate the request in CalculateFareSchema. vehicleTier and ISO pickup datetime are required; return datetime must not precede pickup. The legacy schema accepts an optional distanceKm for compatibility, capped at 5,000 km.
2. Resolve product context in fare.service.ts and product.resolver.ts. Published product records can contribute fleetPrices, fixed package price, distance, per-km override, driver allowance, night charge, nights, upgrades, toll, and rule version.
3. Resolve distance using distance.resolver.ts. The implementation first accepts a positive finite input.distanceKm, then product/dossier distance, catalog distance, override distance, fixed defaults (package 100 km, local 80/120 km, transfer 20 km), then route resolver fallback. See risk R-02: despite schema comments and prior audit claims, this resolver accepts client-supplied distance when present.
4. fare.engine.ts selects the applicable branch: admin/dossier tier base fare; legacy tour package; catalog-backed ride/tour; curated local package; airport transfer; or route strategy.

5. finalizeFare() calculates configured/default night allowance when applicable, adds driver allowance and configured tollAmountInr, applies an eligible promo discount, then computes total, advance, balance and explanatory rules.
6. The service contains an extra database-promo flow: it first calls calculateFare without the promo, then applies the resolved DB promo to baseFare + nightAllowance + driverAllowance. This differs from finalizer subtotal logic because the reapplication subtotal omits tollAmountInr if a toll has been added by finalizer. Verify whether this causes discount/total mismatch for toll-bearing routes.

3. Service-by-service fare behavior

Service/product	Observed rule path	Important boundaries / caveats
Outstation one-way: Sedan/Ertiga/Innova	StandardVehiclePricingStrategy: if active vehicle rate is considered custom, distance × rate; otherwise route.fares[vehicle] from the static route catalogue.	Static route prices can supersede active DB rate when hasCustomRate is false. For dynamic route estimates, static catalogue and dynamic rates can produce different commercial amounts.
Outstation same-day round trip: standard cabs	Uses max(route fare × 1.85, min daily km × rate). Default sameDayRoundMultiplier = 1.85, minKmPerDay = 300.	Round-trip math uses both a route fare and rate-based daily floor. Ensure these rules are intended together and that the route fare source is consistent.
Outstation multi-day: standard cabs	Minimum daily distance × rate; actual distance is also compared. Defaults include 300 km/day and ₹300/day driver allowance.	Calendar days are computed in IST inclusively. Return datetime and long trip constraints need explicit tests.
Tempo Traveller / Force Urbania	GroupCommercialVehicleStrategy. If distance <300 km, force round trip and bill distance ×2; at/above 300 km, billed distance follows requested trip type. Daily allowance defaults to ₹500/day.	300.0 km is a hard boundary: 299.9 km doubles; 300 km does not. Group promos rejected unless configured/explicitly permitted. Some legacy product branches have separate formulas.
Tour packages	Newer product resolver can use tour_packages.fleet_prices as fixed per-tier commercial rates. If no tier price and fixed starting price exists, starting price + upgrade surcharge is a fallback.	Older/static fallback package records also exist. New and legacy package branch behavior needs a canonical precedence contract.
Local sightseeing	Fixed rates for 8hr/80km and 12hr/120km; current backend catalogue values are Sedan ₹1,900/₹2,200; Ertiga ₹2,600/₹2,950; Innova ₹2,850/₹3,100; Tempo ₹5,500/₹6,500; Urbania ₹7,500/₹8,500.	Static local-package amounts can remain after an Admin rate change unless database product records are used. Group vehicles get ₹500 driver allowance in curated branch.
Airport/station transfer	Can use published database transfer route configuration or static transfer match. Static examples include Agra station, Agra airport, and Delhi IGI with per-tier fixed fares.	The matcher uses substring checks and static defaulting. Unknown airport-transfer wording may resolve to an unintended default location/product. Need explicit route/product identity.
Local route (origin == destination)	Route resolver returns an existing local route or synthesizes an 8-hour/80-km sightseeing route. Local route calculator uses catalog fares.	An outstation typo/normalization that makes origin and destination equal can change pricing mode; reject/clarify rather than silently treat as sightseeing.
Promotions	Database promo resolver reads active state, dates, min total, redemption count and allowGroupVehicles. Discount is capped at subtotal.	Redemption count validation is not, by itself, atomic consumption. If redemption limits are strict, redemption must be claimed transactionally at booking/payment confirmation.
Advance and balance	28% of total, rounded to nearest ₹100, minimum ₹500, not more than total. Balance = total - advance.	Client duplicates this formula in multiple pages/helpers, with slight rounding/branch discrepancies. All visible advance amounts should come from the server quote.

Service/product	Observed rule path	Important boundaries / caveats
Night / toll charges	Default night window in backend rules is 20:00–06:00 IST; cab ₹300, Tempo/Urbania ₹500. Product nightChargeInr/nightHaltInr can override. tollAmountInr is added when positive.	Static catalogue comments contain a different night window (22:00–05:00). Must define one policy and verify every calculator branch that sets applyNight.

4. End-to-end request and money flow

- Admin opens /fares, reads current rules, edits min daily km, night allowances, fleet names/capacity/perKm/active and saves.
- Backend admin service writes a new fare_rules version, marks/uses it as active through repository behavior, writes an audit record and triggers a frontend rebuild hook.
- Customer asks for a quote through POST /api/v1/fares/calculate. The backend validates, resolves current product/rule records and calculates the fare.
- Booking draft path resolves the selected published package/local/route, recalculates fare server-side, then persists totals and a fare snapshot/version for historical stability.
- Razorpay checkout is documented to use the stored booking advance amount rather than a customer-submitted amount; webhook verification completes the paid/confirmed transition.

Trust boundary Never accept totalFare, baseFare, advanceAmount, balanceAmount, toll, driver fee, or billable distance from the customer browser as authoritative. Client values are hints at most; resolve the product, distance, active rules and total on the server.

5. High-risk findings and discrepancies

ID / severity	Finding	Evidence / impact	Recommendation
R-01 / Critical	Customer pages have several separate price authorities.	docs/PRICE_SOURCE_MATRIX.md records hardcoded homepage fleet starting fares, static route fares, static package amounts/upgrades, monument transfer prices, and frontend indicative calculations. Customer can see one amount and receive another at quote time.	Introduce a canonical server-provided FleetPriceOption/quote response for every price display. Remove duplicated fare math from components.
R-02 / High	Client-provided distance can win distance resolution.	distance.resolver.ts returns input.distanceKm first when positive; request schema explicitly accepts distanceKm for legacy clients. Prior docs say server derives distance, but implementation contradicts that blanket statement.	Ignore/remove customer-provided distance for billable calculations. Derive from a trusted route/product or vetted routing provider; apply a controlled, audited distance override only for privileged operator workflows.
R-03 / Critical	Static route fares are used when no custom rate is detected.	StandardVehiclePricingStrategy uses route.fares[vehicleId] unless hasCustomRate. Database rate updates may not affect a route when the code sees a default-looking perKm value.	Make route base fare explicitly route distance × active fleet rate plus explicitly modeled surcharge, or designate fixed route price as its own product rule with no ambiguous fallback.
R-04 / High	Advertised prices are stale-prone.	Static SEO/pre-rendered pages, localStorage manifest and	Define which fields are editorial starting prices vs guaranteed

ARENA AI / SK BAGHEL TOUR & TRAVELS / FARE SYSTEM

ID / severity	Finding	Evidence / impact	Recommendation
		frontend data can remain stale after Admin update; repo docs say only one dynamic dossier detail surface clearly uses live package values.	quotes. Version/etag the public quote/catalog response; purge/version caches or revalidate static pages when commercial pricing changes.
R-05 / High	Night rule definitions disagree.	fare.catalogue.ts comment says 22:00–05:00 while night.rules.ts defaults to 20:00–06:00. Admin rule chips may also show one default while engine uses another.	Centralize policy constants and UI schema. Add boundary tests at 19:59, 20:00, 05:59, 06:00 and verify the intended business policy.
R-06 / High	300 km boundary is not uniform across all product types.	Group strategy uses <300; route branch also has its own behavior; package/transfer/local calculators have separate formulas. Distances are sometimes set from package day count or parsed text.	Define rules by product type and service class. Use Decimal/rounding policy; test 299, 299.9, 300, 300.1 and missing return date for each applicable branch.
R-07 / High	DB promo path may not preserve toll in its subtotal.	fare.service.ts applies DB promotion to baseFare + nightAllowance + driverAllowance after an initial finalization. finalizer can add tollAmountInr. Reapplied total can omit toll and overwrite prior promo values.	Use a single calculation pipeline that accepts a validated promo lookup before finalization. Unit-test promo + toll + night + driver combinations.
R-08 / High	Distance fallback is an estimate, not route distance.	route.resolver.ts falls back to a city-level DIST_MAP plus defaults for unknown endpoints; frontend customDistance has a separate geospatial estimate path.	Do not present an estimate as a guaranteed price. Require route/geocoding confirmation or label quote provisional until server validates it.
R-09 / Medium	Tier canonicalization is essential but legacy aliases persist.	Both short IDs (innova, tempo) and canonical tiers (innova-crysta, tempo-traveller) are mapped. Unknown ids in toInternalVehicleId fall back to sedan.	Reject unknown tier IDs at boundaries. Normalize once into canonical enum; do not silently price unknown input as Sedan.
R-10 / Medium	Fare rule activation/version must be transaction-safe.	Admin writes a new rules record and triggers rebuild; correctness depends on repository save/activate semantics and deploy hook success.	Atomically activate exactly one version, preserve rollback, emit audit diff, and surface propagation/rebuild status to Admin. Do not claim customer sync until verified.
R-11 / Medium	Promo redemption limits may race.	Promo validity checks read redemptionCount but no redemption reservation is visible in the inspected resolver. Simultaneous bookings could exceed a hard cap.	Reserve or increment redemptions transactionally at a defined state transition, with idempotency and rollback rules.
R-12 / Medium	Price math is duplicated in customer helper/page code.	advanceOf, getIndicativeBrowseFare, calcFare and multiple route/package/monument components calculate money locally.	Frontend formatting is allowed; business rules should be executed server-side. Indicative prices should be clearly typed/labeled and never used for payment or booking persistence.

6. Important constants observed

Rule	Observed implementation	Engineering note
Fleet defaults	Sedan ₹10/km, Ertiga ₹14/km, Innova Crysta ₹18/km, Tempo Traveller ₹25/km, Force Urbania ₹34/km.	Defaults are not proof of live DB values; inspect active ruleset in deployed environment.
Capacity defaults	Sedan 4, Ertiga 6, Innova 6, Tempo Traveller 12, Urbania 16 seats.	Capacity should be enforced against passenger count server-side and aligned with marketing nomenclature.
Group threshold	Distance <300 km => doubled billable distance; ≥300 km => requested trip-type behavior; driver allowance ₹500/day.	Explicit business rule; not a generic rule for every service.
Standard round trip	Same-day multiplier default 1.85x; multi-day floor 300 km/day, driver allowance ₹300/day.	Confirm whether rate should be corridor fare × multiplier or distance × rate to avoid inconsistent margins.
Night allowance	Cab ₹300; group ₹500; engine default time window 20:00–06:00 IST.	Make UI, API, engine and written fare policy match.
Advance payment	28%, rounded to nearest ₹100, minimum ₹500, capped at full total.	Calculate once in the canonical fare finalizer; payment order uses persisted amount.
Limits	Distance must be >0 and ≤5000 km; return date can be at most 30 days after pickup in fare engine.	Also validate plausible travel duration, allowed date range, passenger count, exact units, finite values and canonical enums.

7. Suggested engineering sequence

- Freeze and document the actual commercial policy, including one-way/round-trip semantics, 300 km threshold, min km/day, night window, driver allowances, toll inclusion, waiting, parking, inter-state permits, taxes, cancellation and refunds.
- Create typed, canonical contracts for FleetTier, TripType, ProductType, FareRuleVersion, RouteQuoteRequest, PackageQuoteRequest and FareQuoteBreakdown. Reject unknown enum values; do not default unknown fleet to Sedan.
- Choose exactly one price model per product: route-per-km, fixed route quote, fixed package by fleet, local package with included usage and extra rates, or transfer tariff. Encode this in product data instead of inferring from overlapping flags.
- Keep the engine pure and deterministic. Do I/O in resolvers; perform pricing in calculators; do shared surcharges/promotions/rounding in one finalizer. Use explicit money rounding and document when rounding occurs.
- Replace all customer price surfaces with server-supplied prices or clear “starting from / indicative” editorial text. Do not show a made-up total where route inputs are incomplete.
- Persist immutable fare snapshot, quote ID, active rule version, chosen product/rule IDs, distance source, billed distance, all charge components and timestamps with booking. Bind checkout to this persisted snapshot.
- Build regression tests around every service × fleet × trip type × boundary. Include promo/toll/night interaction and admin changes. Deploy behind compatibility adapters; compare old and new quotes before deleting old logic.

8. Sources reviewed

- Fare service: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.service.ts>
Main branch code review • 10 Oct 2026 | Internal engineering reference

ARENA AI / SK BAGHEL TOUR & TRAVELS / FARE SYSTEM

- Fare engine: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.engine.ts>
- Fare catalogue: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/fare.catalogue.ts>
- Product resolver: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/resolvers/product.resolver.ts>
- Distance resolver: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/resolvers/distance.resolver.ts>
- Standard vehicle pricing: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/route.calculator.ts>
- Group commercial pricing: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/commercial-group.calculator.ts>
- Fare finalizer: <https://github.com/tarun1sisodia/ArenaAI/blob/main/backend/src/modules/fares/calculators/fare.finalizer.ts>
- Admin fare editor: <https://github.com/tarun1sisodia/ArenaAI/blob/main/admin/src/pages/FaresPage.tsx>
- Price flow audit: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/PRICE_FLOW_AUDIT.md
- Price source matrix: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/PRICE_SOURCE_MATRIX.md
- Customer rendering audit:
https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/CUSTOMER_RENDERING_AUDIT.md
- Migration plan: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/MIGRATION_PLAN.md
- Booking price audit: https://github.com/tarun1sisodia/ArenaAI/blob/main/docs/BOOKING_PRICE_AUDIT.md

9. Worked code traces: how to interpret the rules

This section turns the source findings into small examples. Amounts are not new business commitments. They demonstrate the observed defaults and highlight where current branches or data sources can disagree. Use the exact same inputs in automated tests when reproducing these traces.

Trace A — Group commercial vehicle threshold

Observed implementation: `commercial-group.calculator.ts` tests `input.distanceKm < 300`. Under that boundary it doubles billable distance. At or above 300 km it follows the requested trip type, with a special extra condition for a round-trip request lacking `returnDatetime`. Driver allowance defaults to ₹500 per calendar day.

Input	Calculation	What it shows
Tempo, 299 km, one-way	$598 \text{ km} \times ₹25 + ₹500 = ₹15,450$	Below 300 km, the engine forces double distance.
Tempo, 300 km, one-way	$300 \text{ km} \times ₹25 + ₹500 = ₹8,000$	At exactly 300, <code>< 300</code> is false; no forced doubling.
Urbania, 299.9 km	$599.8 \text{ km} \times ₹34, \text{rounded} + ₹500 = ₹20,893$	Decimal distance must be tested; this is a high-impact cliff.
Urbania, 300 km	$300 \text{ km} \times ₹34 + ₹500 = ₹10,700$	The 0.1 km boundary crossing drops the modeled subtotal sharply.

Audit conclusion

The threshold is implemented, but its customer/commercial suitability is a business decision, not just a coding detail. Ensure the UI explains forced round-trip billing and decide whether the price discontinuity is intentional.

Trace B — Fixed products should not become route formulas

Product	Observed price model	Reasoning
8hr / 80km Agra sightseeing	Fixed local Sedan fare: ₹1,900 in the curated defaults	The package bundles time and distance; it is not automatically $80 \times ₹10$.
Published tour package with <code>fleetPrices</code>	Use the configured price for the selected tier	Package price has precedence; a global per-km rate should not silently rewrite it.
Airport/station transfer	Use published transfer fleet price or the specifically configured per-km branch	Transfers are a separate product type with dedicated resolver/calculator behavior.
Outstation corridor	Route strategy; static route fare can be used when no per-km override branch is selected	A stored corridor price and distance $\times$ global rate are different commercial models.

Trace C — Adding charges and a promo

Illustration for Tempo at 300 km one-way: base ₹7,500; driver allowance ₹500; night allowance ₹500; toll ₹400. The subtotal is ₹8,900. If a valid ₹1,000 promo is active, meets its minimum and redemption rules, and is allowed for this fleet, total becomes ₹7,900. Advance is 28% of ₹7,900 = ₹2,212 rounded to the nearest ₹100 = ₹2,200; balance = ₹5,700.

This illustrates the finalizer order: base and authorized extras form the subtotal, promo discounts the subtotal, then advance/balance are derived from the discounted total. Group vehicles are promo-ineligible unless a qualifying database promo explicitly permits them.

Trace D — Two inconsistencies the test suite must catch

Finding in source	Example impact	Recommended assertion
`resolveTripDistanceKm` accepts valid positive `input.distanceKm` before DB product/catalog distances. Request schema accepts the field for legacy compatibility.	A public quote can be influenced by client distance in code paths where a later clamp does not correct it.	The public request cannot set authoritative distance; server-resolved distance source is returned and tested.
Night helper defaults to 20:00–06:00 IST; `fare.catalogue.ts` comment says 22:00–05:00.	The same pickup time can be described under two different rule windows.	Single versioned rule; 19:59, 20:00, 05:59, 06:00 tested in IST.
Multi-day standard strategy uses the 300 km/day minimum in base math but reports original route distance as `billedKm`.	The price and displayed breakdown can disagree.	For 150 km over 2 days, show actual distance 150 km and billable basis 600 km if that is the charging rule.
Frontend contains static prices and indicative calculations while backend has active rule data.	Homepage, routes, monuments or package details may display a different price from booking.	Price-display parity test compares each binding price surface to a documented server quote/starting-price contract.

10. How to reproduce these examples in code review

- Use the same vehicle tier, trip type, pickup and return datetimes, origin/destination, product ID, active fare-rule version, and promo in every reproduction.
- Trace the call chain: `fare.service.ts` -> `product.resolver.ts` and `distance.resolver.ts` -> `fare.engine.ts` -> selected calculator/strategy -> `fare.finalizer.ts`.
- Write down each intermediate value: trusted distance, route/product, resolved rate, base fare, billable km, driver allowance, night fee, toll, discount, total, advance, and balance.
- Compare the result with `GET /api/v1/fleet`, admin fare rules, the customer display card, the booking snapshot, and checkout amount.
- When a discrepancy appears, classify it as a conflicting business policy, source-of-truth defect, validation defect, implementation bug, or stale presentation/cache issue.

Do not mix “current behavior” with “desired behavior”

The repo currently has legacy and server-side pricing paths coexisting. Mark a proposed formula as a target policy until it is implemented and proven by tests. An audit example is not itself approval of a commercial price.